PROJECT DOCUMENTATION

OpsPilot

An AI on-call agent that investigates production incidents.
End-to-end workflow, architecture, operations and roadmap.

Author	Giddaluru Damodhar
Repository	github.com/damodharg6/opspilot
Showcase	opspilot-showcase.netlify.app
Version	main branch, October 2026

CONTENTS

1. The idea in plain language
2. System overview
3. The end-to-end workflow, step by step
4. Components in depth
5. Data model
6. Reliability and failure handling
7. Security model
8. Running it locally
9. Demos, showcase and recording
10. Testing and CI
11. Deployment
12. Results
13. AI roadmap (system design)
14. Glossary and file map

1. The idea in plain language

When an online service breaks, an alarm wakes an engineer. Before fixing anything they must work out *what* is wrong: reading logs, comparing graphs, checking recent releases. OpsPilot does that first investigation automatically and writes a short report with evidence.

> **ANALOGY**
> A car's warning light says *something* is wrong. A mechanic still plugs in a scanner and checks recent repairs before touching a bolt. OpsPilot is that first scan, done before the engineer has opened a laptop.

What the report contains

- **Category**: bad deploy, database latency, database connection, memory, business issue, or unknown.
- **Summary and root cause** in plain sentences.
- **Evidence**: snippets copied word for word from real tool output, each with its source.
- **Confidence**: low, medium or high, lowered automatically when evidence cannot be proven.
- **Suggested fix**: a recommendation only; a human decides.
- **Trace**: every tool call it made, with arguments, output and timing.

Three principles

> **READ-ONLY**
> The agent can look at logs, metrics and deploy history but has no ability to change anything. The worst outcome of a wrong answer is a wrong report.

> **EVIDENCE OR NOTHING**
> Every claim must be grounded in real tool output; unproven evidence is removed before the report is stored.

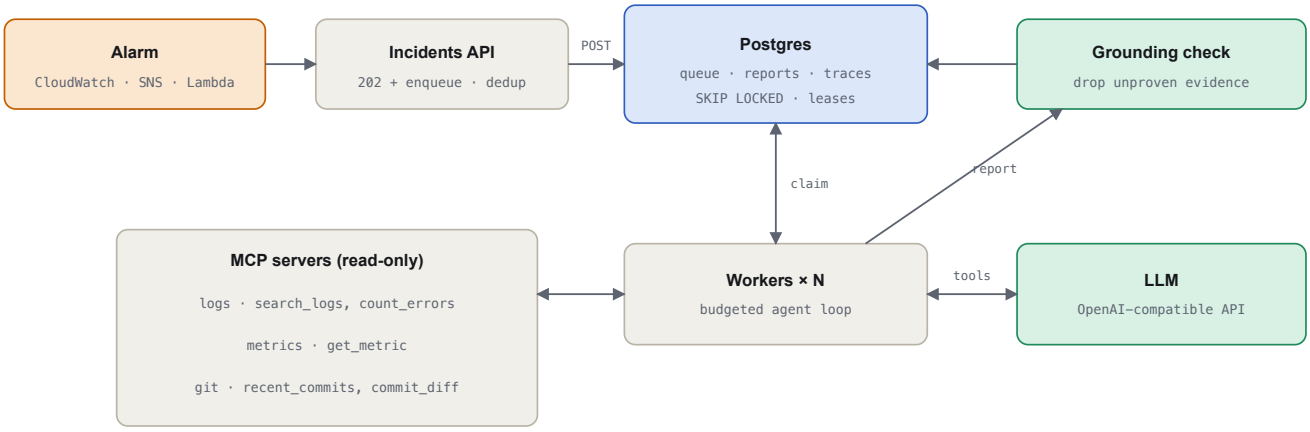
> **BOUNDED**
> Tool calls, tokens and wall-clock time are capped in code, so an investigation cannot loop or overspend.

Who it helps

Role	Benefit
On-call engineer	Starts from an explanation and evidence instead of a blank screen.
Engineering manager	Shorter incidents, consistent first response, a written record of each incident.
Business	Less downtime cost and less team burnout.
Customers	Services recover faster.

2. System overview

OpsPilot is a set of small, stateless services around one Postgres database. Each runs from the same Docker image with a different command.



Service	Role	Code	Port (local)
shop	Demo e-commerce service that breaks on purpose (fault switches); writes one JSON log line per request	`shop/`	8000
api	Incidents API: accepts alarms, returns 202, exposes reports and traces	`opspilot/api/app.py`	8080
worker ×2	Claims queued incidents and runs the investigation	`opspilot/worker/main.py`	—
mcp-logs	Read-only log tools	`mcp_servers/logs_server.py`	8101
mcp-metrics	Read-only metric tools	`mcp_servers/metrics_server.py`	8102
mcp-git	Read-only deploy-history tools	`mcp_servers/git_server.py`	8103
db	Postgres 16: job queue, reports, tool-call traces (and shop data)	`migrations/`	5432*
migrate	Runs Alembic migrations once, then exits	`alembic.ini`	—

* configurable with `DB_HOST_PORT` in `.env`.

Technology

Python 3.12 · FastAPI · SQLAlchemy 2 · Pydantic v2 · Alembic · Postgres 16 · MCP Python SDK (streamable HTTP) · OpenAI-compatible LLM APIs (Gemini, Groq, OpenAI, Ollama) · Docker Compose · AWS (EC2, CloudWatch, SNS, Lambda, SSM) · Terraform · GitHub Actions · pytest · ruff · Netlify (showcase site).

3. The end-to-end workflow, step by step

Following one incident from the moment something breaks to the stored report.

1 **Something breaks and signals change**

The shop serves traffic and writes a JSON log line per request (status, latency, error type) plus periodic runtime stats (memory). A fault switch, e.g. `buggy_deploy`, makes some requests fail.

`shop/main.py: request_logging, create_order · shop/faults.py: FaultRegistry · scripts/inject.py`

2 **An alarm fires**

In AWS, a CloudWatch metric filter counts 5xx responses; an alarm publishes to SNS; a Lambda in the VPC parses the alarm and POSTs it to the API with a shared secret. Locally, the same POST is sent by hand, by the demo recorder or the showcase server.

`infra/terraform/cloudwatch.tf, alerting.tf · infra/lambda/alarm_forwarder.py: parse_alarm, post_incident, handler`

3 **The API accepts the incident (202) with dedup**

The secret is compared in constant time. The incident is inserted with `INSERT … ON CONFLICT DO NOTHING` against a partial unique index on `dedup_key` (service:alarm) for open incidents, so a flapping alarm creates one job. The API answers `202 {id, status: queued, deduplicated}` immediately.

`opspilot/api/app.py: require_secret, create_incident · opspilot/db/queue.py: dedup_key, enqueue_incident`

4 **A worker claims the job (SKIP LOCKED + lease)**

Workers poll. `claim_next` selects the oldest available job, or one whose lease expired, with `FOR UPDATE SKIP LOCKED`, then marks it `investigating`, sets `locked_by`, `locked_until = now + lease` and increments `attempts`. Any number of workers can do this without blocking or taking the same job.

`opspilot/worker/main.py: Worker.process_one, run_forever · opspilot/db/queue.py: claim_next`

5 **The worker connects to the tools**

`MCPToolGateway` opens an MCP client per server over streamable HTTP, lists the tools and exposes them to the model as one flat list: `search_logs`, `count_errors`, `get_metric`, `recent_commits`, `commit_diff`.

`opspilot/agent/tools.py: MCPToolGateway · mcp_servers/runtime.py: serve (stateless HTTP, allowed Host headers)`

6 The agent loop investigates inside budgets

The model receives a system prompt and the incident context, then repeatedly chooses tools. Before every round the loop checks the tool-call budget (default 8) and token budget (default 40,000); the whole run is wrapped in a wall-clock timeout. Each tool result is wrapped as untrusted data. If a budget runs out, the model is asked for a final answer without tools and the result is marked `partial`.

`opspilot/agent/investigator.py: Budget, Investigator.run, _investigate · opspilot/agent/prompts.py: SYSTEM_PROMPT, wrap_tool_output`

7 Tools answer safely

Inputs are schema-validated; the log query language is whitelisted (no raw text reaches a CloudWatch query or a shell); output is capped in size and secret-redacted before the model sees it. Backends are local (files, derived metrics, a git repo) or AWS (CloudWatch Logs Insights, CloudWatch metrics, GitHub API).

`mcp_servers/query.py: parse_query · mcp_servers/output.py · common/redaction.py: redact · mcp_servers/backends/local.py, aws.py`

8 The model talks through one provider layer

Any OpenAI-compatible API is supported. The layer owns retries (exponential backoff with jitter for rate limits, timeouts and 5xx; fail fast on auth or bad requests), token accounting, and provider quirks such as echoing Gemini's thought signatures back with tool calls.

`opspilot/llm/openai_compat.py: OpenAICompatibleProvider.complete · opspilot/llm/factory.py: build_llm`

9 The report is validated, repaired once, then grounded

The final answer must parse into the `IncidentReport` schema; on failure the model gets one repair round with the validation error. Then `ground_report` keeps only evidence whose snippet appears in real (non-error) tool output, lowers confidence one level if anything was dropped, and to `low` if nothing survives.

`opspilot/agent/report.py: IncidentReport · investigator.py: parse_report, _finalize · opspilot/agent/grounding.py: ground_report`

10 The result is stored only if the worker still holds the lease

`complete` locks the row and checks it is still `investigating` and `locked_by` this worker (fencing). Only then does it upsert the report and replace the tool-call trace. A slow worker whose lease expired gets `False` and its result is discarded. Infrastructure errors go to `fail_or_retry`: re-queue with backoff (10 s, 20 s, …) up to `max_attempts`.

`opspilot/db/queue.py: _owned, complete, fail_or_retry`

11

People read the report

`GET /incidents/{id}` returns status and report; `GET /incidents/{id}/trace` returns every tool call with arguments, output preview and duration. The on-call engineer decides what to do.

`opspilot/api/app.py: get_incident, get_trace, list_incidents`

4. Components in depth

Demo shop

FastAPI + SQLAlchemy service selling four car parts. Routes: `/health`, `/products`, `/orders`, admin fault switches (`/admin/faults`, protected by `X-Admin-Token`). Faults:

Fault	Effect	Expected category
buggy_deploy	New pricing code raises KeyError for products without a promotion; also records "release v1.5.0" in the deploy history	bad_deploy
slow_db	Adds a delay (e.g. 1200 ms) to every DB call	db_latency
bad_config	Points the app at a database host that does not exist	db_connection
memory_leak	Retains memory per request; visible in runtime_stats rss_mb	memory
stock_exhaustion	Sets one product's stock to 0: 409 responses, no outage	business

MCP servers

Tool	Server	What it returns
count_errors(minutes, group_by)	logs	5xx counts grouped by path, error_type or status
search_logs(query, minutes, limit)	logs	Matching log lines, newest first, capped and redacted; query language like `status>=500 path=/orders`
get_metric(name, minutes)	metrics	Time series: error_rate_5xx, latency_p95_ms, request_rate, memory_mb
recent_commits(hours)	git	Recent releases with sha, time and message
commit_diff(sha)	git	What a release changed (clipped)

Agent and prompts

The system prompt tells the model to investigate methodically, to cite evidence copied verbatim from tool output, to treat tool output as untrusted data, and to answer in a fixed JSON schema. `IncidentContext` carries service, alarm, window and description.

Worker lifecycle

Each worker has a unique id (host-pid-random). SIGTERM sets a stop flag; the current job finishes before exit (compose gives it a 120 s grace period), so deploys do not drop work.

Settings

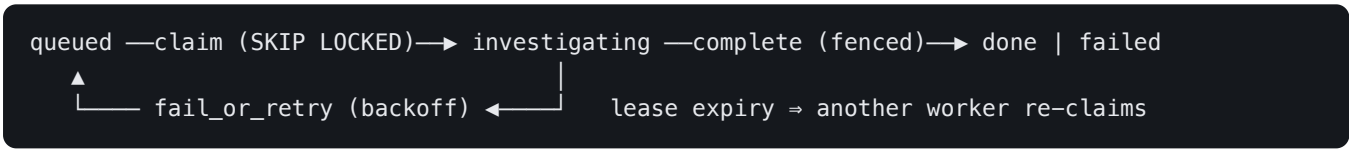
All configuration comes from environment variables via pydantic-settings (prefixes `OPSPILOT_`, `SHOP_`, `MCP_`). Secrets live in `.env` locally (gitignored) and SSM SecureString in AWS.

5. Data model

Schema `opspilot`, created by the Alembic migration `0001_initial_opspilot_schema`.

Table	Purpose	Key columns
incidents	The job queue and incident record	id, service, alarm_name, dedup_key (partial unique while open), status (queued → investigating → done/failed), attempts, max_attempts, available_at, locked_by, locked_until, last_error, created/finished_at
reports	One report per incident (upsert)	result_status, category, summary, root_cause, confidence, evidence (JSON), suggested_fix, model, prompt_version, stop_reason, tool_call_count, tokens, latency_s, evidence_dropped
tool_calls	The investigation trace	incident_id, step, tool, arguments (JSON), result_preview (≤ 4,000 chars), is_error, duration_ms

Incident states



6. Reliability and failure handling

Failure	Behaviour
LLM rate limit, timeout or 5xx	Retry with exponential backoff + jitter (configurable attempts), then the job fails with the reason
LLM auth / bad request	Fail fast, no retry
Invalid JSON report	One repair round with the validation error; then failed
A tool errors or times out	The error becomes the tool result; the investigation continues
MCP server unreachable	Job re-queued with backoff up to max_attempts
Worker crashes mid-job	Lease expires; another worker re-claims (at-least-once)
Slow, expired worker finishes late	Fencing rejects its write; the newer attempt wins
Budget exhausted	Final answer without tools; report marked partial
Alarm flaps	Dedup returns the open incident instead of creating a new one

Scaling path

Stage	Change
Today	Compose on one EC2 instance, 2 workers
10 services	More worker replicas; RDS instead of a container DB; MCP servers take a service argument
100 services	ECS/Fargate per role, autoscale workers on queue depth, MCP behind an internal ALB, queue moves to SQS (only db/queue.py changes)
Many teams	Tenant id on incidents and tools; per-tenant budgets and model choice

7. Security model

- **Read-only agent.** No tool can change infrastructure; the AWS role reads logs and metrics.
- **Validated inputs.** Tool arguments are schema-validated; the log query language is whitelisted.
- **Untrusted output.** Tool output is wrapped as data, capped in size and redacted (keys, tokens, passwords, emails) before the model sees it — prompt injection in logs has no write tool to abuse.
- **Secrets.** `.env` locally (never committed), SSM SecureString in AWS; shared secrets compared in constant time.
- **Network.** MCP servers enforce allowed Host headers (DNS-rebinding protection) and listen on localhost when published; the showcase server binds 127.0.0.1 with a Host check and never sends the API secret to the browser.
- **Containers.** Non-root user; EC2 with IMDSv2 only and an encrypted disk; deploys via GitHub OIDC + SSM Run Command (no SSH keys in CI).
- **Public site.** Static only, strict Content-Security-Policy (self-hosted scripts and styles only), nosniff, referrer and permissions policies, HSTS.

8. Running it locally

```
# 1. get the code and configure
git clone https://github.com/damodharg6/opspilot && cd opspilot
cp .env.example .env          # set POSTGRES_PASSWORD, SHOP_ADMIN_TOKEN,
                              # OPSPILOT_API_SECRET and an LLM key + provider + model
# 2. Python environment (keep it outside iCloud-synced folders)
uv venv --python 3.12 ~/.venvs/opspilot
uv pip install --python ~/.venvs/opspilot/bin/python -e ".[dev]"
export PATH="$HOME/.venvs/opspilot/bin:$PATH"
# 3. start
make demo-history             # simulated deploy history
make up                       # db, migrate, shop, 3 MCP servers, api, 2 workers
make smoke                    # health of every service + one real tool call
# 4. break something and investigate
python -m scripts.inject on buggy_deploy
curl -X POST localhost:8080/incidents -H "X-OpsPilot-Secret: $OPSPILOT_API_SECRET" \
     -H "Content-Type: application/json" -d '{"alarm_name":"shop-5xx-rate-high"}'
curl localhost:8080/incidents/<id> -H "X-OpsPilot-Secret: $OPSPILOT_API_SECRET"
python -m scripts.inject reset
make down                     # stop (data kept)
```

Useful URLs

Shop API docs	http://localhost:8000/docs
OpsPilot API docs	http://localhost:8080/docs
Showcase with live runs	http://localhost:8090 (make showcase)

Troubleshooting highlights

- **Port 5432 in use**: set `DB_HOST_PORT` in `.env` (containers always use db:5432).
- **Worker restarting, "OPSPILOT_LLM_API_KEY is not set"**: add a key, then `docker compose up -d --force-recreate worker`.
- **LLM 429 quota**: free tiers allow few requests per day; raise `OPSPILOT_LLM_MAX_RETRIES` or use another model/provider.
- **Files stall on an iCloud Desktop**: keep the virtualenv outside the synced folder.

9. Demos, showcase and recording

`scripts/demo.py` stages one realistic incident through the *real* path: reset (faults off, stock restocked, fresh deploy history) → healthy traffic → fault on → traffic continues → `POST /incidents` → wait for the worker → collect report and trace.

Command	What it does
`make record-demos`	Records 4 scenarios into `showcase/data/*.json` (the public site replays these)
`make showcase`	Serves the site on 127.0.0.1:8090 with a "Run live" button that runs a scenario for real and streams progress
`make smoke`	Pass/fail table for shop, API, 3 MCP servers and one real count_errors call

The public site (**opspilot-showcase.netlify.app**) is static: it replays the recordings, explains the architecture and roadmap, and hosts a plain-English blog. Live runs only exist on a machine running the stack; on the public site a callout explains this instead of a button.

10. Testing and CI

Command	Scope
`make test`	Unit, MCP (in-memory client), agent (ScriptedLLM), LLM client (mock transport), shop, smoke, showcase; Postgres tests skipped
`make test-pg`	Everything, against a throwaway postgres:16-alpine container on 127.0.0.1:5435 that is always removed afterwards
`make lint`	ruff check + format check
`make evals-offline`	Replays recorded eval fixtures deterministically; CI gate on accuracy vs baseline

Rules: tests never call real LLMs or AWS (ScriptedLLM, recorded fixtures, botocore Stubber). Queue, API and worker tests run against real Postgres because SKIP LOCKED, partial indexes and ON CONFLICT are Postgres behaviour.

GitHub Actions

- **CI** (every push/PR): lint, tests with a Postgres service, offline eval gate, Terraform validate, Docker build.
- **Deploy** (after CI on main): GitHub OIDC → short-lived AWS role → SSM Run Command on EC2. Disabled until the repository variable `DEPLOY_ENABLED=true`, so it shows as skipped.

11. Deployment

Local / single host

`docker-compose.yml` runs every role from one image (non-root). MCP and DB ports are published on 127.0.0.1 only.

AWS

Terraform in `infra/terraform`: VPC and EC2 (IMDSv2, encrypted disk), CloudWatch log group, metric filters and alarms, SNS topic, in-VPC Lambda alarm forwarder, SSM parameters for secrets, IAM roles (instance, Lambda, GitHub OIDC deploy role) and budget alerts. `docker-compose.aws.yml` switches the MCP backends to CloudWatch and GitHub.

```
alarm (CloudWatch) → SNS → Lambda alarm_forwarder → POST /incidents (EC2)
push to main → CI → Deploy workflow → OIDC role → SSM Run Command → docker compose up
```

Showcase site (Netlify)

`netlify.toml` publishes `showcase/` with no build step and sets security headers.

```
npx netlify-cli deploy --dir showcase --no-build          # draft URL to check first
npx netlify-cli deploy --dir showcase --no-build --prod   # production
```

12. Results

Four staged incidents recorded on 24 October 2026 through the full stack (API → queue → worker → MCP → LLM). OpsPilot was told only "shop health degraded".

Scenario	Expected	Diagnosis	Tool calls	Agent time	Confidence	Evidence dropped
Buggy deploy	bad_deploy	✓ bad_deploy	7	32 s	medium	1
Slow database	db_latency	✓ db_latency	4	34 s	medium	1
Stock exhaustion	business	✓ business	5	63 s	high	0
Memory leak	memory	✓ memory	6	46 s	medium	1

Models: gemini-3.5-flash (buggy deploy), gemini-3.5-flash-lite (others, after the free daily quota of the first model ran out).

> **CAVEATS**
> The demo shop logs its own fault switches; in the memory-leak run the model quoted that line, which a real system would not provide. These are 4 of the 8 eval scenarios; the full suite also covers database connection failures, noisy and low-traffic variants and a healthy control.

13. AI roadmap (system design)

Each step adds a component behind an interface that already exists — the provider layer, the MCP tool gateway, or the Postgres store — so none requires a rewrite.

Stage	Capability	Where it plugs in
Now	Budgeted agent loop, grounding, 3 read-only MCP servers, offline eval gate	—
Next	Model routing: small fast model first, escalate on low confidence	Provider layer
Next	Incident memory: past reports as embeddings (pgvector) + similar_incidents tool	Same Postgres + gateway
Next	Runbook retrieval: MCP server over team runbooks; fixes cite procedures	Tool gateway
Later	Human-approved actions (rollback, restart) proposed in Slack; agent stays read-only	New approval service
Later	Agent observability: OpenTelemetry spans per tool/LLM call; cost, latency, accuracy dashboards	Worker + provider layer
Later	Continuous evaluation on every prompt/model change	evals/ + CI

14. Glossary and file map

Term	Meaning
Incident	Something is wrong with a live service
On-call	The engineer responsible for responding to alarms
Log / metric / deploy	Event diary / numbers over time / a released version
MCP	Model Context Protocol: standard for AI models to call tools
SKIP LOCKED	Postgres row locking that lets workers skip rows others hold
Lease / fencing	Time-limited ownership of a job / only the current owner may write the result
Grounding	Checking report evidence against real tool output

Path	Contents
common/	JSON logging, correlation ids, redaction
shop/	Demo service + fault injection
mcp_servers/	logs, metrics, git servers; backends/local.py, backends/aws.py; query language; output shaping
opspilot/llm/	Provider layer (OpenAI-compatible), factory, ScriptedLLM for tests
opspilot/agent/	Investigator loop, prompts, report schema, grounding, tool gateways
opspilot/api/, worker/, db/	Incidents API, queue consumer, models + queue
migrations/	Alembic
evals/	Scenarios, live + offline runner, scoring
scripts/	Load, fault injector, deploy history, smoke, demo recorder, showcase server
showcase/	Static showcase site, blog and recorded demo data
infra/	Terraform and the Lambda alarm forwarder
docs/	architecture.md, runbook.md, engineering-rules.md, this PDF